# UNIT - 4

Flutter is an **open-source UI (User Interface) toolkit** developed by **Google** that allows developers to build applications for **Android, iOS, Web, Windows, macOS, and Linux** using a **single codebase**.
Normally, if you want to build apps for Android and iOS:

- **Android** → Java or Kotlin
- **iOS** → Swift or Objective-C

With **Flutter**, you write the code **once** in **Dart**, and Flutter compiles it to run on multiple platforms.
Flutter uses **Dart**.
Code:

```
void main() {
  runApp(MyApp());
}
```

Example:

```
import 'package:flutter/material.dart';

void main() {
  runApp(
    MaterialApp(
      home: Scaffold(
        appBar: AppBar(
          title: Text("Hello Flutter"),
        ),
        body: Center(
          child: Text("Welcome"),
        ),
      ),
    ),
  );
}
```

Main components :

## 1. Widgets

Everything in Flutter is a **Widget**.Widgets are **immutable**, meaning they cannot be changed after they are created. If the UI changes, Flutter creates a new widget.

Examples: text , button, image,icon,row, column

## 2. MaterialApp

The root widget for a Material Design application.

It manages: theme , navigation

### 3. Scaffold

Provides the basic structure of a screen.

Ex: Home screen, appbar,body

4. AppBar
Displays the top application bar.

Ex: AppBar(

  title: Text("My App"),

)

5. Body
Contains the main content of the screen.

body: Center(

  child: Text("Welcome")

)

# Features of Flutter

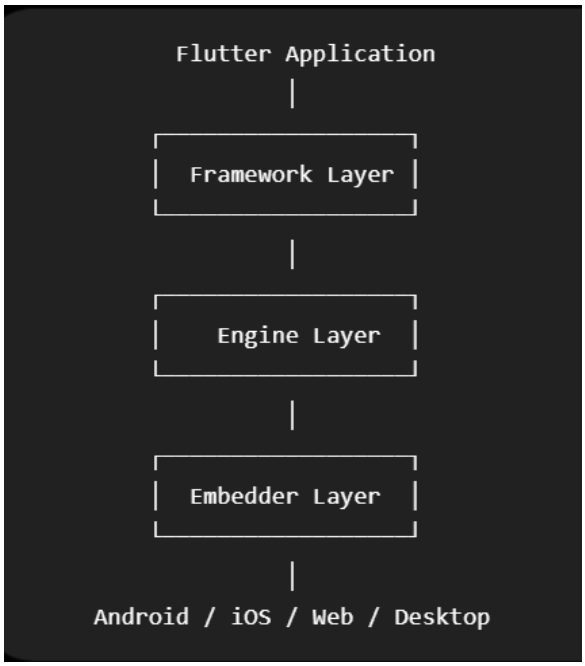
- Single codebase for multiple platforms
- Fast development with **Hot Reload**
- High performance
- Rich collection of built-in widgets
- Attractive UI with smooth animations
- Easy integration with native Android and iOS features

# Flutter Architecture

Flutter architecture is divided into **three main layers**:

### 1. Framework Layer

The **Framework Layer** is written in **Dart**. It is the part developers interact with when building Flutter apps. **It contains:** Widgets, Material Design widgets, Cupertino (iOS-style) widgets, Navigation, Animations




## 2. Engine Layer

The **Engine Layer** is written mainly in **C++**. It converts the widget tree into pixels that appear on the screen.

## Responsibility

- Draws the UI.
- Executes Dart code.
- Handles graphics and animations.
- Manages images and fonts.

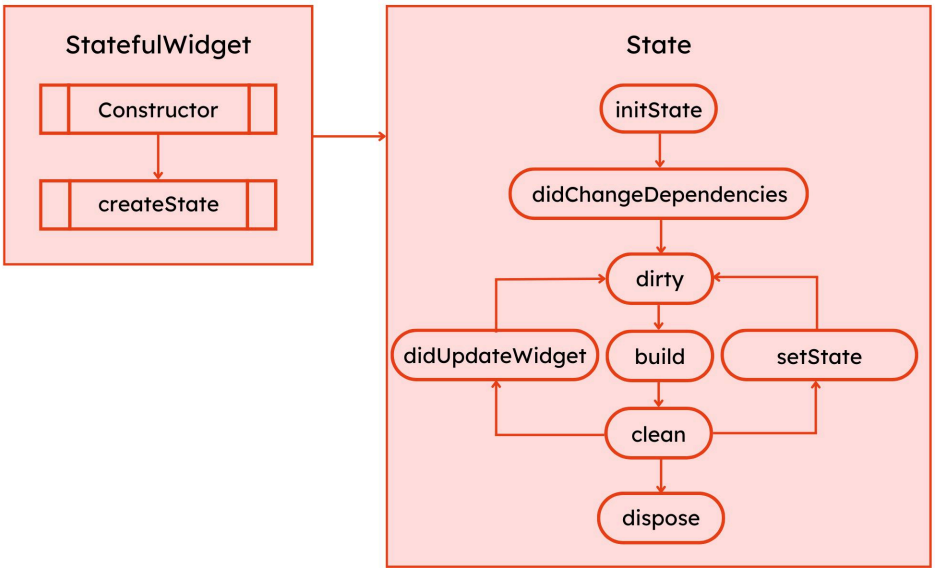
## 3. Embedder Layer

The **Embedder Layer** connects Flutter to the underlying operating system.

Examples:

Android,iOS,Windows,macOS, Linux, Web

| Feature | Widget | Element |
|---|---|---|
| Definition | Blueprint/configuration of the UI | Object that manages a widget in the tree |
| Nature | Immutable | Mutable |
| Purpose | Describes what should appear | Connects widget to render object and manages updates |
| Lifetime | Recreated whenever UI changes | Usually reused and updated |
| Created by | Developer | Flutter framework |
| Stores | UI configuration | Widget location, lifecycle, and state connection |




| Method | Purpose | Called |
|---|---|---|
| `createState()` | Creates the State object | Once |
| `initState()` | Initialize data/resources | Once |
| `didChangeDependencies()` | Respond to dependency changes | After `initState()` and when dependencies change |
| `build()` | Builds the UI | Initially and after updates |
| `setState()` | Notifies Flutter to rebuild | Whenever state changes |
| `didUpdateWidget()` | Handles new widget configuration | When parent passes new properties |

| | | |
|---|---|---|
| `deactivate()` | Widget temporarily removed | Before disposal or reinsertion |
| `dispose()` | Releases resources | Once, before destruction |

Flutter Installation

**Operating System:** 64-bit Linux

**Disk Space:** At least **2.8 GB** (excluding IDE and Android SDK)

**Tools:** `git`, `curl`, `wget`, `unzip`

**IDE (Optional):** Android Studio or Visual Studio Code

Flowchart

Update Linux



Install Required Packages

Download Flutter SDK

Add Flutter to PATH

Reload PATH

flutter --version



flutter doctor



Install Android Studio & Plugins



flutter create my_app



flutter run

**StatelessWidget**

```
class WelcomeScreen extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return Center(
      child: Text("Welcome to Flutter"),
    );
  }
```

```
}
```

**StatefulWidget**

```
class CounterApp extends StatefulWidget {
  @override
  State<CounterApp> createState() => _CounterAppState();
}

class _CounterAppState extends State<CounterApp> {
  int count = 0;

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      body: Center(
        child: Text("$count"),
      ),
      floatingActionButton: FloatingActionButton(
        onPressed: () {
          setState(() {
            count++;
          });
        },
        child: Icon(Icons.add),
      ),
```

```
    );
  }
}
}
```

| Feature | Stateless Widget | Stateful Widget |
|---|---|---|
| Definition | Widget whose data does not change after creation | Widget whose data can change during runtime |
| State | No internal state | Has mutable state |
| Nature | Immutable | Mutable state |
| UI Update | Only when parent rebuilds or configuration changes | Updates whenever `setState()` is called |
| Performance | Slightly faster and simpler | Slightly more overhead due to state management |
| Lifecycle | Simple (`build()`) | More complex (`createState()`, `initState()`, `build()`, `dispose()`, etc.) |
| Memory Usage | Lower | Higher than Stateless Widgets |

| | | |
|---|---|---|
| Examples | `Text`, `Icon`, `Image`, `Container` | Counter, Login Form, Checkbox, Switch |

**Dart** was chosen for Flutter because it provides **high performance**, **native compilation**, **Hot Reload**, **object-oriented programming**, **strong type safety**, **easy syntax**, and **excellent support for asynchronous programming**. These features make it suitable for building fast, responsive, cross-platform applications.

The **`main()` function** is the entry point of every Dart application. Execution begins from `main()`, and in Flutter it typically calls `runApp()` to start the application.

**Comments** are used to explain code and improve readability. Dart supports **single-line (`//`)**, **multi-line (`/* */`)**, and **documentation (`///`)** comments.

**Variables** are named memory locations used to store data such as integers, strings, and booleans. They make applications dynamic by allowing data to be stored and updated.

**Package imports** use the `import` keyword to include built-in libraries or external packages. For example, `import 'package:flutter/material.dart';` provides Material Design widgets required to build Flutter applications.

Ex:

```
/* Importing the Dart core library

 (optional, imported by default)

*/

import 'dart:core';

void main() {

  var name = "Bhavana";

  final int age = 21;

  const String college = "XYZ Engineering College";

  print("Name: $name");

  print("Age: $age");

  print("College: $college");
```

```
}


import 'dart:async';

Future<void> fetchData() async {

  print("Downloading data...");

 await Future.delayed(Duration(seconds: 3));

 print("Download Complete");

}

void main() {

  fetchData();


  print("App is Running...");

}
```

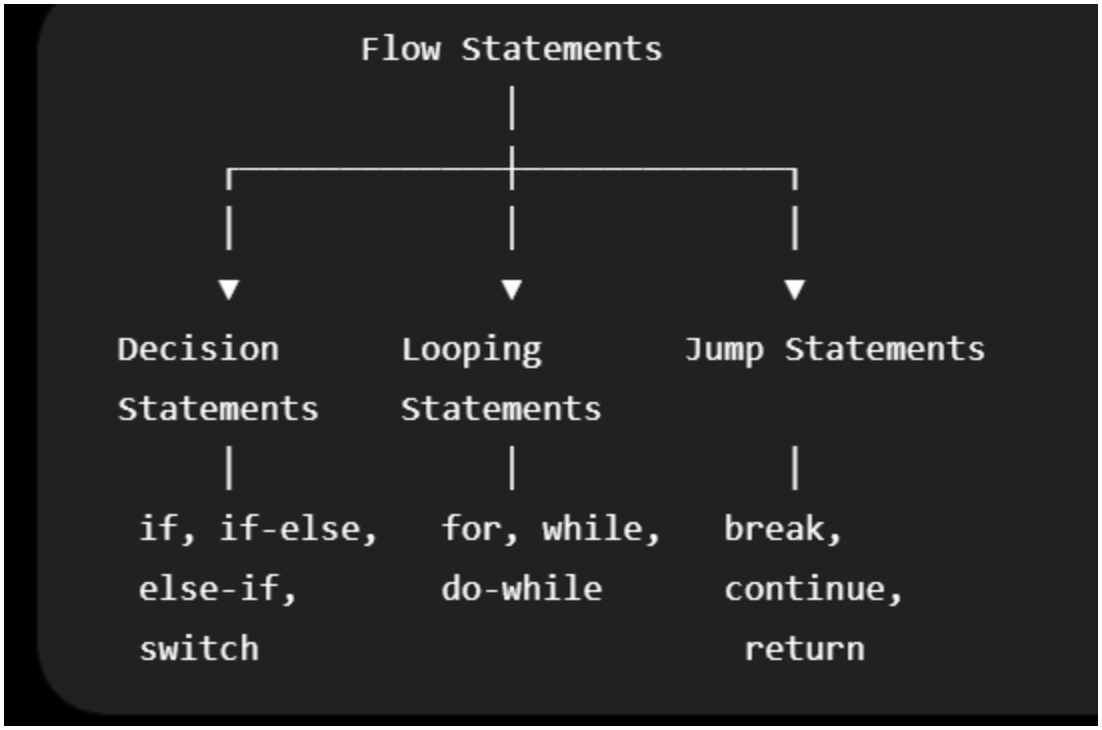
Dart uses **two compilation techniques**:

1. **Just-in-Time (JIT) Compilation** – Used during **development**.
2. **Ahead-of-Time (AOT) Compilation** – Used during **production**.

**Just-in-Time (JIT)** compilation compiles Dart code **while the application is running**.

**Ahead-of-Time (AOT)** compilation converts Dart code into **native machine code before the application runs**.

There are **three main types** of flow statements in Dart:


Flow Statements
Decision Statements
Looping Statements
Jump Statements
if, if-else, else-if, switch
for, while, do-while
break, continue, return